

Labyrinthe DFS — Rapport succinct

Présentation du projet

Ce projet consiste à réaliser un solveur de labyrinthe en langage C utilisant l'algorithme DFS (*Depth First Search*).

Le programme lit une map depuis un fichier, vérifie sa validité puis recherche un chemin entre une entrée E et une sortie S.

Le projet est organisé en deux dossiers principaux :

srcs/ : contient les fichiers sources du programme ainsi que les différentes fonctions du solveur.

maps/ : contient plusieurs labyrinthes de test sous forme de matrices.

Dans les maps :

'1' représente un mur

'0' représente un chemin accessible

'E' représente l'entrée

'S' représente la sortie

Les fonctions ont été séparées dans plusieurs fichiers afin d'améliorer la lisibilité et l'organisation du code.

Fonctionnement du programme

Le programme suit les étapes suivantes :

- 1) Lecture de la map depuis un fichier.
- 2) Copie de la map en mémoire sous forme de matrice.
- 3) Vérification de la validité de la map.
- 4) Exécution de l'algorithme DFS afin de trouver un chemin.
- 5) Affichage : de la map originale, puis du chemin trouvé avec des couleurs pour améliorer la visibilité.

Algorithme utilisé : DFS

L'algorithme DFS (*Depth First Search*) explore le labyrinthe de manière récursive.

Le principe est : partir de l'entrée, puis visiter une direction, continuer tant qu'un chemin est possible, revenir en arrière lorsqu'une impasse est rencontrée (*backtracking*).

Les cases déjà visitées sont marquées afin d'éviter les boucles infinies.

Cet algorithme ne garantit pas toujours le chemin le plus court, mais il permet de trouver rapidement une solution valide dans un labyrinthe.

Choix réalisés

Plusieurs choix ont été faits durant le développement :

- utilisation de matrices de caractères pour représenter simplement le labyrinthe
- séparation du code en plusieurs fichiers afin de rendre le projet plus clair
- utilisation de fonctions dédiées pour : la lecture, les vérifications, l’affichage, le DFS, ajout de couleurs dans le terminal afin de mieux visualiser le chemin trouvé.

Problèmes rencontrés

Plusieurs difficultés ont été rencontrées pendant le projet :

- gestion correcte de la récursivité
- éviter les boucles infinies lors de l’exploration
- compréhension du retour arrière (*backtracking*)
- affichage du bon chemin sans colorer toutes les cases visitées
- gestion des limites de la map pour éviter les erreurs mémoire
- Le débogage du DFS a représenté la partie la plus complexe du projet.

Mode d’emploi

(Dans le README.md)

Exécution

(Dans le README.md)

Exemple d’utilisation

(Dans le README.md)

Discussion et conclusion

Ce projet a permis de mieux comprendre :

- les matrices en C
- la gestion mémoire
- la récursivité
- les algorithmes de parcours de graphe

Le DFS est relativement simple à implémenter mais demande une bonne compréhension du backtracking.

Une amélioration possible serait d’utiliser un algorithme BFS (*Breadth First Search*) afin de toujours trouver le chemin le plus court.

DING Alexandra — L1-X